Code-based Masking: From Fields to Bits Bitsliced Higher-Order Masked SKINNY

John Gaspoz and Siemen Dhooghe

COSIC, ESAT, KU Leuven, Leuven, Belgium
{john.gaspoz,siemen.dhooghe}@esat.kuleuven.be

Abstract. Masking is one of the most prevalent and investigated countermeasures against side-channel analysis. As an alternative to the simple (*e.g.*, additive) encoding function of Boolean masking, a collection of more algebraically complex masking types has emerged. Recently, inner product masking and the more generic code-based masking have proven to enable higher theoretical security properties than Boolean masking. In CARDIS 2017, Poussier *et al.* connected this "security order amplification" effect to the *bit-probing model*, demonstrating that for the same shared size, sharings from more complex encoding functions exhibit greater resistance to higher-order attacks. Despite these advantages, masked gadgets designed for code-based implementations face significant overhead compared to Boolean masking. Furthermore, existing code-based masked gadgets are not designed for efficient bitslice representation, which is highly beneficial for software implementations. Thus, current code-based masked gadgets are constrained to operate over words (*e.g.*, elements in $\mathbb{F}_{2^k}$), limiting their applicability to ciphers where the S-box can be efficiently computed via power functions, such as AES. In this paper, we address the aforementioned limitations. We first introduce foundational masked linear and non-linear circuits that operate over bits of code-based sharings, ensuring composability and preserving bit-probing security, specifically achieving t-Probe Isolating Non-Interference (t-PINI). Utilizing these circuits, we construct masked ciphers that operate over bits, preserving the security order amplification effect during computation. Additionally, we present an optimized bitsliced masked assembly implementation of the SKINNY cipher, which outperforms Boolean masking in terms of randomness and gate count. The third-order security of this implementation is formally proven and validated through practical side-channel leakage evaluations on a Cortex-M4 core, confirming its robustness against leakages up to one million traces.

Keywords: Code-based Masking · Masking · Probe Isolating Non-Interference · Probing Security · Side-channel Analysis · Software

1 Introduction

Side-Channel Analysis (SCA) poses a significant threat to cryptographic primitive implementations in both software and hardware. In such attacks, an adversary exploits physical leaked information from the device, such as power consumption, electromagnetic radiation, computation time, etc., in order to extract some secret information. One of the most prevalent and investigated countermeasures against SCA –and more specifically against power analysis– is *masking*. In its essence, a d^{th}-order masking leverages an encoding function which randomizes a sensitive value x into a collection of $d+1$ shares $(x_0, \ldots, x_d)$ such that the secret can be retrieved from the whole set, while an incomplete set of at most d shares does not provide any information on x. Additionally, masking transforms the original operations over the unmasked values into masked gadgets which operate over

the shares. Boolean masking [GP99, CJRR99] is the most common masking and utilizes the XOR as the encoding function such that $x = x_0 + \ldots + x_d$. Thanks to its simple encoding, Boolean masking benefits from masked gadgets with small overhead while ensuring theoretically d^{th} order security in the probing model [ISW03]. Besides Boolean masking, other masking schemes relying on more algebraically complex encoding functions have been progressively introduced such as inner product masking (IPM) [BFG+17], polynomial masking [GM11], (orthogonal) direct sum masking (ODSM) [BCC+14] or the generic code-based masking (GCM) [WMCS20, WCG+22] . A direct benefit from using a more complex encoding function is a higher security order for a small shared state size. This security enhancement was first discussed over the inner product masking scheme in the work of Wang *et al.* [WSY+16] and later observed in Balasch *et al.* [BFG+17]. Inner product masking leverages an encoding function which randomizes a secret value x using inner product between random values and a public (or private) vector. More specifically, given a vector of random values and a selected public vector $\bar{L} = (L_0, \ldots, L_d)$ with $L_i \in \mathbb{F}_{2^k} \backslash \{0\}$, the $d+1$ sharing of a secret value $\bar{x} = (x_0, \ldots, x_d)$ is computed such that $x = \sum_{i=0}^{d} L_i x_i$. In their work, the authors observed that specific parameters of the public vector $\bar{L}$ would influence the perceived information theoretic metric. Intuitively, the inner product between the randomness and the public vector will XOR multiple random bits to each secret bit introducing the observed security increase in contrast to Boolean masking where only one bit of each randomness is added to a secret bit. This security observation for IPM was later connected by Poussier *et al.* [PGS+17] to direct sum masking, where the encoding of a secret value $X \in \mathbb{F}_2^k$ is constructed on the bit-level with $Y \in \mathbb{F}_2^m$ the randomness as $Z = X\mathbf{G} + Y\mathbf{H} \in \mathbb{F}_2^n$ using their respective generator matrices $\mathbf{G}$ and $\mathbf{H}$, to the minimal dual distance of the code $\mathbf{H}$ denoted as $d_{\mathbf{H}}^{\perp}$. The security of these more complex masking schemes now distinguishes a *word-level* and a *bit-level* security, the latter of which reflects this security order amplification under the bit-probing model, while the former maps to the standard probing model which relates to the number of shares.

1.1 Previous Works

In their work [WSY+16], Wang *et al.* first introduced an hybrid masking scheme known as "Boolean matrix product masking", a variant of inner product masking, enabling bitsliced implementation of a more algebraically complex encoding. The authors establish security against d^{th}-order attacks within the word probing model, requiring $n \geq 2d + 1$ shares. While, they observe a security amplification effect in the encoding components of the scheme, they leave as an open question whether this security amplification would persist during private computation on real-world devices. In a follow-up work, Wang *et al.* [WYS19] introduced the first code-based masking scheme with provable secure order amplification, using tensor product re-masking for secure multiplication. While bit-probing security is formally proven, composability is supported only by informal analysis. Their claims remain theoretical, with no practical side-channel evaluations to confirm security on real devices. Later, Wang *et al.* [WMCS20] introduced a generic algorithm to compute on data masked with linear codes. In an attempt to reduce the implementation overhead arising from code-based masking, they propose a "new amortization" technique enabling to process multiple sensitive variables in parallel. However, the multiplication gadget fails to preserve the order amplification, which has been shown to result in leakages during practical implementations, as observed by Wu *et al.* [WCG+22]. Moreover, Wu *et al.* findings highlight that the "cost amortization" technique can lead to a security level reduction in practice.

A recent work from Gaspoz *et al.* [GD24] details the design of a multiplication gadget tailored for inner product masking which preserves bit t-SNI security (*e.g.*, the security order amplification) during computation. The theoretical security of the gadget is formally proven and their analysis is complemented with a practical side-channel leakage evaluation

on a real-world device. However, the gadget is restricted to inner product masking and its generalization to code-based masking cannot be applied straightforwardly. Additionally, their practical implementation highlights computational overhead for which the author left optimization as future work.

1.2 Limitations

The benefits introduced by these more complex masking schemes come with limitations and drawbacks. The first drawback of code-based masking relates to its non-negligible overhead compared to Boolean masking counterpart. Obviously, due to the use of more algebraically complex encoding functions, operations within masked gadgets become heavier. Additionally, software Boolean masked implementations benefit from bitsliced implementations allowing the computation of multiple instances of a cryptographic primitive (*e.g.*, S-Boxes) in parallel which can lead to a large performance increase. However, the optimization relies on Boolean circuits which makes code-based masked circuits unfit for the bitsliced optimization. Indeed, existing masked gadgets for more complex masking scheme solely operate over words (*e.g.*, with elements $\mathbb{F}_{2^k}$). As a result, this limits their usage solely to ciphers in which the S-Box can be efficiently computed via power functions, hence the usual implementation of AES as reference cipher in previous existing works [BFG+17, CGM20, WMCS20]. While any S-box can be constructed from the composition of power permutations and affine transformations [APB+23], such a construction comes with non-negligible overhead. As a result, no masked implementation of ciphers operating on individual bits (such as KECCAK, PRESENT, SKINNY, etc.) has been implemented using IPM or GCM, let alone with bitsliced optimization. Finally, as observed in Wu *et al.* [WCG+22], existing non-linear masked gadgets for GCM [WMCS20] fail to preserve bit-probing security which translates into a loss of the security order amplification during private computations. Thus, it remains a challenge to provide full-fledged masking schemes which maintain the security order amplification (under linear leakages) of the sharing throughout secure computations.

1.3 Contributions

In this work, we overcome the aforementioned limitations and provide construction of code-based masking applied to bitsliced implementations enabling, for the first time (to our knowledge) the application of code-based masking to ciphers operating on individual bits while maintaining the bit probing security order during secure computation. Our contributions can be summarized as follows.

First, we introduce foundational masked linear and non-linear circuits that operate over code-based sharings, preserving the bit-probing security level of the masking and ensuring composability. Specifically, we first detail the construction of a masked Toffoli gate operating on code-based inputs from which we then derive two linear gates: a linear ADD masked gate directly instantiated from the Toffoli gate and a SWAP gate that is derived from the ADD gate. We demonstrate that these gadgets achieve t-Probe Isolating Non-Interference (t-PINI) within the bit-probing model.

Second, we demonstrate how masked ciphers can be constructed from these basic circuits. This allows for code-based masked implementations of ciphers operating over bits while preserving the security order amplification effect throughout secure computations. Consequently, this enables enhanced security with a reduced shared state size.

Third, we present an optimization specifically designed for a bitsliced masked assembly implementation of the SKINNY cipher, which outperforms theoretically equivalently secure Boolean masking implementations in terms of randomness and gate count. We formally prove the third-order security of our optimized implementation. Finally, we validate the theoretically predicted third-order security of the code-based bitsliced implementation

on a Cortex-M4 core by performing practical side-channel leakage evaluation where no significant (univariate, bivariate, or trivariate) leakages were found for up to one million traces.

2 Preliminaries

2.1 Notations

We use lower-case letters for elements of a finite field $\mathbb{F}_q$. With $+$, we denote the field addition in some binary field $\mathbb{F}_2^n$. The masking of a secret $x \in \mathbb{F}_2^k$ is represented as a row vector $\bar{x} \in \mathbb{F}_2^n$ and the notation $\bar{x}[i]$ (resp., $x[i]$) represents the i^{th} bit in the sharing $\bar{x}$ (resp. plaintext x). Matrices are denoted by bold capital letters, with $\mathbf{M}[i, j]$ representing the element at row i and column j. Given two matrices $\mathbf{A}$ and $\mathbf{B}$, $[\mathbf{A}; \mathbf{B}]$ denotes the row-wise concatenation of $\mathbf{A}$ and $\mathbf{B}$. The same notation applies to vectors $\bar{x}$ and $\bar{y}$.

Definition 1 (Dual Distance)**.** Let $C \subseteq \mathbb{F}_{2^k}^n$ be a linear code. The dual distance of C, denoted $d_C^\perp$, is the smallest Hamming weight among all non-zero codewords in the dual code $C^\perp$. Formally,

$$d_C^\perp = \min\left\{\mathrm{wt}(x) \mid x \in C^\perp,\, x \neq 0\right\},$$

where $\mathrm{wt}(x)$ is the Hamming weight of x.

2.2 Code-Based Masking

Here we consider the most general case of code-based masking namely generalized code-based masking (GCM) introduced in [WMCS20] which enables the general representation of multiple masking types such as Boolean masking, inner product masking, polynomial masking, (orthogonal) direct sum masking, etc., where each specific masking imposes conditions on the generating matrices and their resulting codewords.

Given $X \in \mathbb{F}_q^k$ and $Y \in \mathbb{F}_q^m$ referred to as sensitive variable and random variable, respectively, we obtain the shared vector

$$Z = X\mathbf{G} + Y\mathbf{H} \in \mathbb{F}_q^n \tag{1}$$

where $n \geq k + m$ and $\mathbf{G}$ and $\mathbf{H}$ are two generator matrices of codes C and D, respectively, such that $C \cap D = \{0\}$. Similarly, the shared vector can be constructed as

$$Z = [X, Y]\mathbf{A} \in \mathbb{F}_q^n \tag{2}$$

using the matrix $\mathbf{A} = [\mathbf{G}; \mathbf{H}] \in \mathbb{F}_q^{(k+m)\times n}$. Namely, $\mathbf{G}$ is the $k \times n$ upper part matrix of $\mathbf{A}$ generating the codes C from its multiplication with the secret variable X. Similarly, $\mathbf{H}$ is the $m \times n$ lower part matrix encoding the randomness Y into codes D.

2.3 Security Definitions

We detail the security notions and probing models used throughout the paper. We first start with the definition of the probing model introduced by Ishai *et al.* [ISW03] to formally define the security order of a masking scheme. The model establishes the security order of a masked circuit by allowing an adversary to observe up to t wires, demonstrating that no information from the sensitive variables can be recovered.

Definition 2. (Probing Model) A t-order probing adversary $\mathcal{A}$ can learn a set of t wires (referred to as probes) within a circuit. A circuit is considered t-probing secure if any subset of at most t probes can be perfectly simulated by a simulator $\mathcal{S}$ without access to the input shares of the circuit. In other words, the distribution of the probed values remains independent of any secret values.

Next, we refine the probing model based on the granularity of the probed information by specifying two types of probes used by the adversary, namely:

- **Word-probing model**: each probe captures an ℓ-bit word at a time, meaning an ℓ-bit variable (*e.g.*, a byte in $\mathbb{F}_{2^8}$) leaks in its entirety and the (word) security order is represented as t_w. This model is evaluated under the usual probing model from Ishai *et al.* [ISW03].
- **Bit-probing model**: each probe reveals a single bit element from the encoding and the (bit) security order is represented as t_b. This model can be evaluated under the bounded moment leakage model from Barthe *et al.* [BDF+16].

Naturally, word-level security order cannot surpass the one at bit-level. The side channel-security order of code-based masking is directly connected to the security order in the bit probing model. Hence, code-based masking may benefit from a higher security order than Boolean masking for the same number of shares. The work from Poussier *et al.* [PGS+17] connects the bit-level security of direct sum masking and inner product masking to the maximal dual distance of the code D, denoted as $d_D^\perp$ and the security to be exactly $t_b = d_D^\perp - 1$. Note that such a security increase is only guaranteed under the assumption of linear leakage functions [WSY+16, BFG+17] which fortunately dominate the leakages observed on real world devices [PGS+18]. This framework remains valid as long as the two codes C and D are complementary. Note however that, in the case of GCM, the constraints on the complementarity of two codes are not enforced anymore (*e.g.*, $n \geq k + m$). This relaxation allows for achieving a security order even larger than the dual distance [WMCS20, Proposition 5]. Thus, the security benefit of code-based masking is directly tied to the careful selection of an appropriate generating matrix $\mathbf{H}$ for the code D. Among the possible generator matrices, two main selection parameters are used. The first selection parameter for an ideal matrix, as mentioned above, is the dual distance $d_D^\perp$. However, given different choices of the code D with the same dual distance, a smaller number of nonzero codewords of minimal weight will reduce both information-theoretic metrics, namely the signal-to-noise ratio and the mutual information [CGC+21]. With these metrics in mind, the generation of an optimal generator matrix remains, however, based on exhaustive or heuristic approaches (by using subfield representation with trace-orthogonal bases) [CGC+21, CLGR24]. Fortunately, optimal linear code matrices for two- and three-share masking in $\mathbb{F}_{2^4}$ and $\mathbb{F}_{2^8}$ are available in Cheng *et al.*'s work and their GitHub[1] repository [CGC+21] or in Magma [BCP97] database.

Lemma 1. *For a uniform encoding $\bar{x}$ of $x \in \mathbb{F}_2^k$ with generating matrix $\mathbf{A} = [\mathbf{G}; \mathbf{H}]$, all sets of (at least) $t = d_D^\perp - 1$ bits are jointly uniform distributed.*

Gadget compositions are evaluated through the Probe-Isolating Non-Interference (PINI) notion by Cassiers and Standaert [CS20] for which we first define simulatability.

Definition 3 (Simulatability [CS20])**.** Let $P = \{p_1, ..., p_\ell\}$ be a set of ℓ probes of a gadget C and C_P the tuple of values of the probes for an execution of C. Let $I = \{(i_1, j_1), ..., (i_k, j_k)\} \subset \{0, ..., d\} \times \{0, ..., m-1\}$ be a set of input bits of C. A simulator is a randomized function $\mathcal{S} : \mathbb{F}_2^k \to \mathbb{F}_2^\ell$. The set of probes P can be simulated with the set of input bits I if there exists a simulator $\mathcal{S}$ such that for any inputs $x_{*,*}$, the distributions $C_P(x_{*,*})$ and $\mathcal{S}(x_{i_1,j_1}, ..., x_{i_k,j_k})$ are equal, where the probability is over the random coins in C and $\mathcal{S}$.

The above definition defines the security game in terms of a simulation game. This framework is extended to PINI security, where we define the information available to the simulator. In addition, we consider the notion over bit-probes.

[1] https://github.com/Qomo-CHENG/OC-IPM

Definition 4 (PINI [CS20])**.** Let G be a gadget over d shares and P a set of t_0 bit-probes on wires of G (called internal probes). Let A be a set of t_1 share indices. G is t-PINI if for all P and A such that $t_0 + t_1 \leq t$, there exists a set B of at most t_0 bit-share indices such that probes on the set of wires $P \cup y_A$ can be simulated with the wires $x_{A \cup B}$, with x_A denoting the wires with bit-shares in the set A.

To clarify, since we will be focused on working over bit-probing security, we consider a share as a bit in the encoding of the shared value. In addition, we need to use the definition of a uniform sharing and of a uniform function. Namely, a (Boolean or code-based) sharing of a secret variable is called uniform if the share vector is uniformly distributed over the set of all possible sharings of that secret. A uniform function is then a function which maps a uniform input sharing to a uniform output sharing.

3 Gadgets over Code-based Masking

In this section, we present a series of gadgets constructed over code-based masking. We first introduce a general masked Toffoli gate (TOF) that operates on code-based inputs, and we demonstrate its t-PINI security at the bit-level. Next, we construct two linear gates derived from specific cases of the TOF gate: a linear ADD masked gate (directly derived from the TOF) and a SWAP gate, which is further derived from the ADD gate.

Any cipher can be decomposed into a series of linear and non-linear layers. The gadgets in this work are used such that linear layers can be efficiently implemented using our masked basic ADD and SWAP gadgets and that non-linear layers can be implemented using a combination of both the ADD and the TOF gadgets based on the observations of the work by Daemen *et al.* [DDE+20] who provide implementations of the known 3, 4, and 5-bit S-boxes in terms of Toffoli gates.

3.1 Toffoli Gate

We describe here the general overview for computing Toffoli gate from code based masking. For that, we define a gadget named $\mathrm{TOF}(\bar{x}, \bar{y}, \mathbf{A}, u, v, w)$ (see Algorithm 1) which takes as input some shares $\bar{x} = (x_0, \ldots, x_{n-1})$ and $\bar{y} = (y_0, \ldots, y_{n-1}) \in \mathbb{F}_2^n$ from a code-based masking with generating matrices $\mathbf{G} = (\mathbf{1}_k, 0_k) \in \mathbb{F}_2^{k \times n}$ and a matrix $\mathbf{H} \in \mathbb{F}_2^{m \times n}$ where $\mathbf{1}_k$ and $\mathbf{0}_k$ denote the identity and zero $k \times k$ matrix, respectively. The function outputs shares $\bar{z} = (z_0, \ldots, z_{n-1}) \in \mathbb{F}_2^n$ such that its decoding results into the value $z \in \mathbb{F}_2^k$ with $z[u] = x[u] + y[v]y[w]$, namely the multiplication and addition of bits within the input parameters $x, y \in \mathbb{F}_2^k$. In a nutshell, the algorithm will refresh each binary shares of the cross product $y[v]y[w]$ using some bit from a secure code-based encoding of zero and successively add these refreshed shares to $\bar{z}[u]$. On a high level, the algorithm can be divided into three main steps.

1. In the first step, input share $\bar{y}$ is duplicated and refreshed with a secure code-based encoding of zero in order to obtain the sharing $\bar{y}'$. This refresh guarantees the mutual independence of the input shares.

2. In the second step, using the public generating matrix $\mathbf{A} = [\mathbf{G}; \mathbf{H}]$, we construct two sets of binary shares S_v and S_w. The set of shares S_v (resp. S_w) contains $t + 1$ binary elements $s_i \in \bar{y}$ (respectively $s_j \in \bar{y}'$), whose XOR results in the unmasking of $y[v]$ (respectively, $y[w]$).

3. In the third step, our goal is to iteratively compute and re-mask the cross products between shares $s_i \in S_v$ and $s_j \in S_w$. Specifically, for each of these cross products, we generate a code-based encoding of zero, denoted $CB(0)$. From this encoding, all bits except the bit at index u are added to $\bar{x}$ while the bit at index u is used as uniform

randomness to re-mask the cross product $s_i s_j$. Thus, once all binary shares have been processed, this results into the secure addition of $y[v]y[w]$ to $\bar{x}[u]$. Correctness is preserved at the completion of this step since the addition of the re-masked cross products completes the encodings of zero which were distributed to $\bar{x}$.

Note that, the sets S_v and S_w are solely introduced to ease the notation used in the pseudo-algorithms and proofs. These sets identify the bits to be refreshed with a CB(0): specifically, the bits $\bar{y}[i]$ where $\mathbf{A}[i, v] == 1$ and $\bar{y}'[i]$ where $\mathbf{A}[i, w] == 1$, processed iteratively.

Algorithm 1 (TOF) - Toffoli Gate

Input: $\bar{x}, \bar{y} \in \mathbb{F}_2^n, u, v, w, \mathbf{A} \in \mathbb{F}_2^{n \times n}$
Output: $\bar{z} \in \mathbb{F}_2^n$ such that $z = \bar{z}\mathbf{A}$ where $z[u] = x[u] + y[v]y[w]$ and $z[i_{\neq u}] = x[i_{\neq u}]$

```
▷ Copy and refresh shares of y
ȳ' = ȳ + CB(0) ∈ F_2^n
▷ Generate the set S_v of binary shares of y[v]
S_v = {}
for i = 0 < n where A[i, v] == 1 do
    S_v ← S_v ∪ {ȳ[i]}
end for
▷ Generate the set S_w of binary shares of y[w]
S_w = {}
for i = 0 < n where A[i, w] == 1 do
    S_w ← S_w ∪ {ȳ'[i]}
end for
▷ Re-mask cross products and sum them to x̄[u]
z̄ = x̄
for s_i ∈ S_v do
    for s_j ∈ S_w do
        r̄^{s_ij} = CB(0) ∈ F_2^n
        z̄[ℓ] = z̄[ℓ] + r̄^{s_ij}[ℓ], ∀ℓ ≠ u < n
        t_i = s_i s_j + r̄^{s_ij}[u]
        z̄[u] = z̄[u] + t_i
    end for
end for
```

3.1.1 Example

For the ease of understanding, we provide an example of Algorithm 1. Given the generating matrices

$$\mathbf{G} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}, \qquad \mathbf{H} = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

and $x = (a, b, c, d)$ and $y = (e, f, g, h) \in \mathbb{F}_2^4$ two 4-bit unshared values with a (resp. e) the least significant bit, we obtain the sharing $\bar{x} = (x_0, x_1)$ (resp. $\bar{y} = (y_0, y_1)$) with $x_i = (a_i, b_i, c_i, d_i)$ (resp. $y_i = (e_i, f_i, g_i, h_i)$) $\in \mathbb{F}_2^4$ defined as follows

$$\begin{aligned} a_0 &= a + b_1 + c_1 + d_1 & a_1 \\ b_0 &= b + a_1 + c_1 + d_1 & b_1 \end{aligned}$$

$$
\begin{aligned}
c_0 &= c + a_1 + b_1 + d_1 & \qquad c_1 \\
d_0 &= d + a_1 + b_1 + c_1 & \qquad d_1
\end{aligned}
$$

$$
\begin{aligned}
e_0 &= e + f_1 + g_1 + h_1 & \qquad e_1 \\
f_0 &= f + e_1 + g_1 + h_1 & \qquad f_1 \\
g_0 &= g + e_1 + f_1 + h_1 & \qquad g_1 \\
h_0 &= h + e_1 + f_1 + g_1 & \qquad h_1
\end{aligned}
$$

Let us assume we want to compute the function TOF with parameters ($u = 3, v = 0, w = 2$):

$$
\begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix}, \begin{pmatrix} e \\ f \\ g \\ h \end{pmatrix} \rightarrow \begin{pmatrix} a \\ b \\ c \\ d + eg \end{pmatrix}
$$

First, we make a copy and refresh with a code-based encoding of zero. We obtain the sharings

$$
\begin{aligned}
\bar{x} &= (a_0, b_0, c_0, d_0, a_1, b_1, c_1, d_1) \\
\bar{y} &= (e_0, f_0, g_0, h_0, e_1, f_1, g_1, h_1) \\
\bar{y}' &= (e'_0, f'_0, g'_0, h'_0, e'_1, f'_1, g'_1, h'_1)
\end{aligned}
$$

Next, we aim to re-mask the cross products between the set S_v of binary shares $s_i \in \bar{y}$ of the secret bit $y[v]$ (*e.g.*, the shares such that $\sum_i s_i = y[0] = e$) and the set S_w of binary shares $s_j \in \bar{y}'$ of the secret bit $y[w]$ (*e.g.*, the shares such that $\sum_j s_j = y[2] = g$). Specifically, we focus on the cross products of $y[v]y[w] = (e_0+f_1+g_1+h_1)(g'_0+e'_1+f'_1+h'_1)$. The re-masking of one cross product (*e.g.*, $e_0 g'_0$) works as follows. First, an encoding of zero $\bar{r}^0 = CB(0)$ is generated as

$$
\begin{aligned}
\alpha_0^0 & \qquad \alpha_1^0 \\
\beta_0^0 & \qquad \beta_1^0 \\
\gamma_0^0 & \qquad \gamma_1^0 \\
\delta_0^0 & \qquad \delta_1^0
\end{aligned}
$$

From this encoding of zero, the bit share $\bar{r}^0[u]$ (*e.g.*, δ_0^0) is used as uniform randomness to re-mask $e_0 g'_0$ as $e_0 g'_0 + \delta_0^0$. Next, to ensure correctness, all bits of the sharing of zero $\bar{r}^0$ except $\bar{r}^0[u]$ are added to $\bar{z}$. We end up with the temporary output sharing $\bar{z}$.

$$
\begin{aligned}
s_0 &= a_0 + \alpha_0^0 & \qquad s_1 &= a_1 + \alpha_1^0 \\
w_0 &= b_0 + \beta_0^0 & \qquad w_1 &= b_1 + \beta_1^0 \\
y_0 &= c_0 + \gamma_0^0 & \qquad y_1 &= c_1 + \gamma_1^0 \\
t_0 &= d_0 & \qquad t_1 &= d_1 + \delta_1^0
\end{aligned}
$$

Finally, the re-masked cross product $e_0 g'_0 + \delta_0^0$ is added to $\bar{z}[u]$ (*e.g.*, $\bar{z}[3] = t_0 = d_0$). This process is repeated for every cross product. Thus, we obtain the output shares $\bar{z}$

$$
s_0 = a_0 + \sum_{i=0}^{i=15} \alpha_0^i \qquad\qquad s_1 = a_1 + \sum_{i=0}^{i=15} \alpha_1^i
$$

$$w_0 = b_0 + \sum_{i=0}^{i=15} \beta_0^i \qquad w_1 = b_1 + \sum_{i=0}^{i=15} \beta_1^i$$
$$y_0 = c_0 + \sum_{i=0}^{i=15} \gamma_0^i \qquad y_1 = c_1 + \sum_{i=0}^{i=15} \gamma_1^i$$
$$t_0 = d_0 + eg + \sum_{i=0}^{i=15} \delta_0^i \qquad t_1 = d_1 + \sum_{i=0}^{i=15} \delta_1^i$$

which correctly decodes to $z = (a, b, c, d + eg)$.

3.1.2 Securely Generating $CB(0)$

The code-based masked gadgets all rely on the generation of code-based maskings of zero used to refresh a shared value. We will demonstrate how such a secure encoding of zero $CB(0)$ can be computed. Our construction is based on the "Free encoding of Zero" security property defined in the work of Belaïd *et al.* [BCRT23, Definition 12] which we slightly adapt such that it works with code-based maskings.

Definition 5 (Free Encoding of Zero [BCRT23])**.** Let G be a gadget without input and a single n-shared output z where any set of k bits are uniformly distributed. G is said to be t-free if for every set W of (internal and output) probed wires of G with $|W| \leq t$, there exists a set V of cardinality $|V| \leq |W|$ such that for every set $O \subsetneq [1:n]/V$, any set of $k - |W|$ bits of $z|_O$ is uniformly distributed and mutually independent of the probed values on W and the outputs $z|_V$.

One way to instantiate such a t-free code-based encoding of zero is from Ishai *et al.*'s work [IKL+13] where $t + 1$ regular $CB(0)$ are XORed together. Note that an adversary probing only the secure $CB(0)$ gadget does not recover any secret information, as at least one probe should be placed on a masked sensitive data. Thus, only a $(t-1)$-free gadget is needed for t-bit probing security.

3.1.3 Correctness and Security Proofs

In this section, we present formal proofs establishing the correctness and security of the TOF gadget (Algorithm 1). We show that the gadget has compositional security in the t-Probe-Isolating Non-Interfering (t-PINI) framework under the bit-probing model.

Lemma 2. *For inputs $\bar{x} = [x, r_x]\mathbf{A}, \bar{y} = [y, r_y]\mathbf{A} \in \mathbb{F}_2^n$ (the encoding of a secret x (resp. y) $\in \mathbb{F}_2^k$ using some randomness r_x (resp. r_y) $\in \mathbb{F}_2^m$), $\mathbf{A}$ the public generator matrix used to encode a secret and randomness to a code based masking, and indices u, v, Algorithm 1 (TOF) gives the output $\bar{z} \in \mathbb{F}_2^n$ such that its decoding results in $z \in \mathbb{F}_2^k$ with $z[u] = x[u] + y[v]y[w]$ and $z[i_{\neq u}] = x[i_{\neq u}]$.*

Proof. Following Algorithm 1, we start with the equation of the output and find the following equalities

$$\begin{aligned}
\bar{z}[u] &= \bar{x}[u] + \sum_{s_i \in S_v} \sum_{s_j \in S_w} s_i s_j + \sum_{s_i \in S_v} \sum_{s_j \in S_w} \bar{r}^{s_{ij}}[u] \\
&= [x, r_x]\mathbf{A}[*, u] + \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \mathbf{A}[i, v]\mathbf{A}[j, w](\bar{y}[i]\bar{y}'[j]) + \sum_{s_i \in S_v} \sum_{s_j \in S_w} [0, r_{s_{ij}}]\mathbf{A}[*, u] \\
&= x[u] + [0, r_x]\mathbf{A}[*, u] + y[v]y[w] + \sum_{s_i \in S_v} \sum_{s_j \in S_w} [0, r_{s_{ij}}]\mathbf{A}[*, u]
\end{aligned}$$

$$= x[u] + y[v]y[w] + [0, r_x]\mathbf{A}[*, u] + \sum_{s_i \in S_v} \sum_{s_j \in S_w} [0, r_{s_{ij}}]\mathbf{A}[*, u]$$
$$= x[u] + y[v]y[w] + [0, r_x + \sum_{s_i \in S_v} \sum_{s_j \in S_w} r_{s_{ij}}]\mathbf{A}[*, u]$$

Similarly, for any other bit index $\ell \neq u$ we obtain:

$$\begin{aligned}\bar{z}[\ell] &= \bar{x}[\ell] + \sum_{s_i \in S_v} \sum_{s_j \in S_w} \bar{r}^{s_{ij}}[\ell] \\ &= [x, r_x]\mathbf{A}[*, \ell] + \sum_{s_i \in S_v} \sum_{s_j \in S_w} [0, r_{s_{ij}}]\mathbf{A}[*, u] \\ &= x[\ell] + [0, r_x + \sum_{s_i \in S_v} \sum_{s_j \in S_w} r_{s_{ij}}]\mathbf{A}[*, u]\end{aligned}$$

Thus, $\bar{z} = z + [0, r_x + \sum_{s_i \in S_v} \sum_{s_j \in S_w} r_{s_{ij}}]\mathbf{A}$ meaning that $\bar{z}$ is a code-based masking of z. □

Theorem 1. *Algorithm-1 (TOF) is t-PINI with t the bit-probing security order of the code-based encoding.*

Proof. We start from the observation that the sharing $\bar{x} = CB(x)$ is t-probing secure by Lemma 1 and the composable security of the creation of $CB(0)$ is shown in Section 3.1.2.

Let $\Omega = (\mathcal{I}, \mathcal{O})$ be a set of t observations on the internal and on the output bits, respectively, where $|\mathcal{I}| = t_0$ such that $t_0 + |\mathcal{O}| \leq t$. We construct a perfect simulator $\mathcal{S}$ of the adversary's bit-probes, which can make use of at most t bits from the same share indices of $\bar{x}$ and $\bar{y}$. Let $w_1, \ldots, w_t$ be the set of probed wires. We classify the internal wires in the following groups:

1. $\bar{x}[i], \bar{y}[i]$
2. $\bar{r}_i[j]$
3. Any wire of $CB(0)$ for remasking $\bar{y}$
4. Any wire of $CB(0)$ for $\bar{r}_i[j]$
5. $\bar{y}'[i] = \bar{y}[i] + r_j[i]$
6. $\bar{z}[i] = \bar{z}[i] + r_j[i]$
7. $t_i = \bar{y}[i]\bar{y}'[j] + r'_i[j']$
8. $\bar{z}[i] = \bar{z}[i] + t_i$
9. Any sum of $\bar{z}[i]$ to create $\bar{z}[\ell]$

We define a set of indices I with $|I| \leq t$, such that the values of the probed wires, denoted w_h with $h = 1, \ldots, t$, can be perfectly simulated given only the knowledge of $\bar{x}[i]_{i \in I}$ and $\bar{y}[i]_{i \in I}$. The set is constructed as follows.

- Initially I is empty.
- For every wire as in the group (1), (2), (5), (6) and (7) add i to I.
- For every wire as in the group (3) and (4), from Definition 5, we know there is a set V of cardinality at most the number of observed bits in $CB(0)$. Add V to I.
- For every wire as in the group (8), and (9) nothing is added to I.

Since the adversary is allowed to make at most t probes, we have that $|I| \leq t$.
We now show that the simulator $\mathcal{S}$ is able to simulate any internal wire w_h as follows.

1. For each observation as in category (1), then $i \in I$ and by definition the simulator has access to $\bar{x}[i]$ and $\bar{y}[i]$. Hence the values are perfectly simulated.
2. For each observation as in categories (2)-(4), the simulator assigns a random and independent value to the observed wire. For categories (3) and (4), we know from Definition 5, that the non-probed bits of $CB(0)$ act independent from the probed ones and that they still follow the uniformity notion from Definition 1. Thus, the other output bits of $CB(0)$ still act as uniform and independent randomness.
3. For each observation as in category (5), by definition the simulator has access to $\bar{y}[i]$. The value $\bar{r}_j[i]$ is either freshly generated or repeated if it was already probed in categories (2)-(4). Hence the value $\bar{y}'$ can be computed as in the real algorithm.
4. For each observation as in category (6), by definition the simulator has access to $\bar{x}[i]$ and $\bar{y}[i]$. The value $\bar{r}_j[i]$ is either freshly generated or repeated if it was already probed in categories (2)-(4). Hence the value $\bar{z}[i]$ can be computed as in the real algorithm.
5. For each observation as in category (7) or (8), t_i and $\bar{z}[i]$ can be computed as in the real algorithm given that the simulator either already has the elements to compute them or it assigns a random and independent value to t_i and compute $\bar{z}[i]$ as in the algorithm.
6. For each observation as in category (9), if any $\bar{z}[i]$ or any component of the sum forming $\bar{z}[\ell]$ has already been probed, then the simulator has access to $\bar{x}[i]$ and the respective randomness, and can simulate it as in the previous steps. Otherwise, from viewing the bits $\bar{z}[i]$, a probe in this category views only a sum of jointly uniform random bits $\bar{r}_j[i]$ from separate and independent code-based encoding of zero. From Lemma 1, and the restriction of the adversary to maximally place t probes, it follows that the probed bits of a uniform random $CB(0)$ does not reveal any secret and can be simulated as joint uniform randomness. Thus, all the $\bar{z}[i]$ in the sum of $\bar{z}[\ell]$ are jointly uniform random and can be simulated as such.

Since the simulation of category (9) added no extra information to the simulator, we also prove that the output of the gadget can be simulated using no additional information. □

As a side note, from the above proof it can be seen that the gadget is both Strong Non-Interferent (SNI) and PINI since no shares are given to the simulator when probing the output.

We then show that the gadget achieves $n-1$ word probing PINI security, with n the number of word-sized shares. That is, the gadget not only provides a nontrivial security increase in the bit-probing security model but also achieves the expected lower bound on the word-probing security order. To be clear, although each probe reveals a full word element, certain internal computations still operate on individual bits.

Theorem 2. *Algorithm-1 (TOF) is t-PINI with $t = n-1$, the word-probing security order of the code-based encoding.*

Proof. The proof follows a similar structure to Theorem 1, but relaxes the constraints on the probed information, allowing the adversary to access full word elements instead of individual bits. Let $\Omega = (\mathcal{I}, \mathcal{O})$ be a set of $t = n-1$ observations on the internal and on the output bits, respectively, where $|\mathcal{I}| = t_0$ such that $t_0 + |\mathcal{O}| \leq t$. We construct a perfect simulator $\mathcal{S}$ of the adversary's word-probes, which can make use of at most t shares from the same share indices of $\bar{x}$ and $\bar{y}$. Let $w_1, \ldots, w_t$ be the set of probed wires. We classify the internal wires in the following groups:

1. x_i, y_i
2. Any shares of $CB(0)$ for $\bar{r}^{s_{ij}}$ or for remasking $\bar{y}$
3. $y'_i = y_i + CB(0)_i$
4. $\bar{z}[i] = \bar{z}[i] + r_j[i]$
5. $t_i = \bar{y}[i]\bar{y}'[j] + r'_i[j']$
6. $\bar{z}[i] = \bar{z}[i] + t_i$
7. Any sum of $\bar{z}[i]$ to create $\bar{z}[\ell]$

We define a set of indices I with $|I| \leq t$, such that the values of the probed wires, denoted w_h with $h = 1, \ldots, t$, can be perfectly simulated given only the knowledge of $x_{i\in I}$ and $y_{i\in I}$. The set is constructed as follows.

- Initially I is empty.
- For every probe as in the group (1)-(5) add i to I.
- For every wire as in the group (6) and (7) nothing is added to I.

Since the adversary is allowed to make at most t word-probes, we have that $|I| \leq t$.

We now show that the simulator $\mathcal{S}$ is able to simulate any internal element w_h as follows.

1. For each observation as in category (1), then $i \in I$ and by definition the simulator has access to x_i and y_i. Hence the values are perfectly simulated.
2. For each observation as in category (2), the simulator assigns a random and independent value to the observed element.
3. For each observation as in category (3), by definition the simulator has access to y_i. The value $CB(0)_i$ is either freshly generated or repeated if it was already probed in categories (2). Hence the value y'_i can be computed as in the real algorithm.
4. For each observation as in category (4), by definition the simulator has access to x_i and y_i. The value $\bar{r}_j[i]$ is either freshly generated as a full word random value or repeated if it was already probed in category (2). Hence the value $\bar{z}[i]$ can be computed as in the real algorithm.
5. For each observation as in category (5) or (6), t_i and $\bar{z}[i]$ can be computed as in the real algorithm given that the simulator either already has the elements to compute them or it assigns a full word random and independent value to t_i and compute $\bar{z}[i]$ as in the algorithm.
6. For each observation as in category (7), if any $\bar{z}[i]$ or any component of the sum forming $\bar{z}[\ell]$ has already been probed, then the simulator has access to x_i (and consequently to $\bar{x}[i]$ as well) and the respective randomness, and can simulate it as in the previous steps. Otherwise, from viewing the bits $\bar{z}[i]$, a probe in this category views only a sum of jointly uniform random shares $CB(0)_j$ from separate and independent code-based encoding of zero. From the restriction of the adversary to maximally place $t = n - 1$ probes, it follows that the probed element of a uniform random $CB(0)_j$ does not reveal any secret and can be simulated as joint uniform randomness. Thus, all the $\bar{z}[i]$ in the sum of $\bar{z}[\ell]$ are jointly uniform random and can be simulated as such.

Since the simulation of category (7) added no extra information to the simulator, we also prove that the output of the gadget can be simulated using no additional information. □

3.2 ADD Gate

We continue with the construction of a gadget for linear operations. For that, we define a gadget named $\texttt{ADD}(\bar{x}, \bar{y}, \mathbf{A}, u, v)$ (see Algorithm 2) which takes as input some shares $\bar{x} = (x_0, \ldots, x_{n-1})$ and $\bar{y} = (y_0, \ldots, y_{n-1}) \in \mathbb{F}_2^n$ from a code-based masking with generating matrices $\mathbf{G} = (\mathbf{1}_k, 0_k) \in \mathbb{F}_2^{k \times n}$ and a matrix $\mathbf{H} \in \mathbb{F}_2^{m \times n}$. The function outputs shares $\bar{z} = (z_0, \ldots, z_{n-1}) \in \mathbb{F}_2^n$ such that its decoding results into the value $z \in \mathbb{F}_2^k$ with $z[u] = x[u] + y[v]$, namely the addition of two bits within the input parameter $x, y \in \mathbb{F}_2^k$. Naturally, this gate can be directly derived from the TOF gate by setting the input parameters v and w to be identical. However, this approach would require the redundant computation of the square operation on a single sensitive bit. Therefore, we propose an optimized algorithm specifically designed for addition. In a nutshell, the algorithm follows similar steps as Algorithm 1 but only requires the re-masking of one set of shares (*e.g.*, the set S_v such that $\sum_i s_i = y[v]$).

Algorithm 2 (ADD) - Two Bits Addition Over Two Encodings

Input: $\bar{x}, \bar{y} \in \mathbb{F}_2^n, u, v \in \mathbb{Z}, \mathbf{A} \in \mathbb{F}_2^{n \times n}$
Output: $\bar{z} \in \mathbb{F}_2^n$ such that $z = \bar{z}\mathbf{A}$ where $z[u] = x[u] + y[v]$ and $z[i_{\neq u}] = x[i_{\neq u}]$
▷ Copy and refresh shares of x
$\bar{y}' = \bar{y} + CB(0)$
▷ Generate the set S_v of binary shares of $y[v]$
$S_v = \{\}$
for $i = 0$ **to** n where $\mathbf{A}[i, v] == 1$ **do**
  $S_v \leftarrow S_v \cup \{\bar{y}'[i]\}$
end for
▷ Re-mask binary shares and sum them to $\bar{x}[u]$
$\bar{z} = \bar{x}$
for $s_i \in S_v$ **do**
  $\bar{r}^{s_i} = CB(0)$
  $\bar{z}[\ell] = \bar{z}[\ell] + \bar{r}^{s_i}[\ell], \forall \ell \neq u$
  $t_i = s_i + \bar{r}^{s_i}[u]$
  $\bar{z}[u] = \bar{z}[u] + t_i$
end for

3.3 SWAP Gate

We introduce one last gadget named $\text{SWAP}(\bar{x}, \bar{y}, \mathbf{A}, u, v)$. This function takes as input the shares $\bar{x}$ and $\bar{y} \in \mathbb{F}_2^n$ from code-based masking. The output is a set of shares $\bar{z} = (z_0, \ldots, z_{n-1}) \in \mathbb{F}_2^n$, which decodes to the value $z \in \mathbb{F}_2^k$. The result ensures that $z[u] = y[v]$ and $z[\ell] = x[\ell]$ for all other indices ℓ. Essentially, this means that the two bits $x[u]$ and $y[v]$ are swapped. The two-bit swap process is detailed in Algorithm 3 and is executed using two sequential calls to the basic gadget ADD. Thus, this gadget can be seen as a special case of the ADD gate which could as well be constructed using only the TOF gate.

3.4 Algorithmic Efficiency

Table 1 illustrates the algorithmic efficiency of the code-based masked gadgets TOF, ADD and SWAP as well as the cost of generating the secure $CB(0)$. For each gadget, the cost induced by the generation of $CB(0)$ is indicated in a gray color. This is coloured since the overall algorithmic efficiency of the gadgets is primarily dominated by the construction

Algorithm 3 (SWAP) - Bits Swap

Input: $\bar{x} \in \mathbb{F}_2^n, \bar{y} \in \mathbb{F}_2^n, u, v \in \mathbb{Z}, \mathbf{A} \in \mathbb{F}_2^{n \times n}$
Output: $\bar{z} \in \mathbb{F}_2^n$ such that $z = \bar{z}\mathbf{A}$ where $z[u] = x[v]$ and $z[i_{\neq u}] = 0$
▷ Compute $\bar{t}$ such that $t[u] = x[u] + y[v]$ and $t[i_{\neq u}] = x[i_{\neq u}]$
$\bar{t} = ADD(\bar{x}, \bar{y}, u, v)$
▷ Compute output $\bar{z}$
$\bar{z} = ADD(\bar{t}, \bar{x}, u, u)$

cost of this secure $CB(0)$. Therefore, developing an optimized construction of a secure t-bit probing secure $CB(0)$ is left as an interesting future work.

Table 1: Algorithmic cost in the number of XOR gates, AND gates, and fresh random bits of Algorithm-1 (TOF), Algorithm-2 (ADD) and Algorithm-3 (SWAP). We denote t the bit-probing security level of the gadget, k the field size and n the size of the codeword.

	#ANDs	#XOR's	#Random bits
Toffoli	$(t+1)^2$	$(t+1)^2(n+1) + n + ((t+1)^2+1) \times CB(0)$	$(t+1)^2 m + m + ((t+1)^2+1) \times CB(0)$
ADD	-	$(t+1)(n+1) + n + (t+2) \times CB(0)$	$m(t+2) + (t+2) \times CB(0)$
SWAP	-	$2 \times ADD$	$2 \times ADD$
$CB(0)$	-	$kt(t-1)$	kt

4 Application to SKINNY

In Section 3, we introduced basic circuits designed for securely computing any function. Section 3.1.3 then established the t-order security of these components and their composability for arbitrary circuits. Building on this foundation, we now demonstrate the practical application of code-based encoding for a complete cipher operating on bits. Specifically, we detail the construction of an optimized ARMv7-M assembly third-order secure masked implementation of the SKINNY cipher evaluated on a ARM Cortex-M4 core. We begin by describing the SKINNY cipher. Following this, we present an optimized masked version of the Q_{294} quadratic class used in the SKINNY S-Box decomposition, which requires fewer gates and less randomness than its Boolean masking counterpart. Finally, we verify the security of our optimized implementation and discuss its algorithmic and practical performance in comparison to both the Boolean masked version and the version based on the general algorithms introduced in Section 3.

4.1 SKINNY

SKINNY [BJK+16] is a tweakable block cipher family intended for efficient hardware and software implementations working over 64-bit (or 128-bit) blocks and a tweakey of size scalar its block size. The state is organized as a square matrix of 4-bit (or 8-bit) cells. The cipher (illustrated in Figure 1) operates over a number of rounds determined by the block and tweakey sizes, with each round consisting of five operations performed in sequence: `SubCells`, `AddConstants`, `AddRoundTweakey`, `ShiftRows` and `MixColumns`. The S-box S of SKINNY is given by the lookup table `c6901a2b385d4e7f` and belongs to the cubic class C_{223}^4. We use the same decomposition as described in the work of Shahmirzadi *et al.* [SM21] $S = A_3 \circ \mathcal{Q}_{294}^4 \circ A_2 \circ \mathcal{Q}_{294}^4 \circ A_1$. All affine functions are bit-permutations up to addition by a constant. The affine functions are defined as

$$A_1 : (a,\ b,\ c,\ d) \mapsto (a+1,\ d+1,\ b+1,\ c+1), \qquad (\texttt{feba7632dc985410})$$

$$A_2 : (a,\ b,\ c,\ d) \mapsto (d,\ c,\ b,\ a), \qquad (\texttt{084c2a6e195d3b7f})$$
$$A_3 : (a,\ b,\ c,\ d) \mapsto (b+1,\ a+1,\ c+1,\ d+1). \qquad (\texttt{fdecb9A875643120})$$

The quadratic function $\mathcal{Q}_{294}^4$ is given by

$$\mathcal{Q}_{294}^4 : (a,\ b,\ c,\ d) \mapsto (a + bd,\ b + cd,\ c,\ d)\,.$$

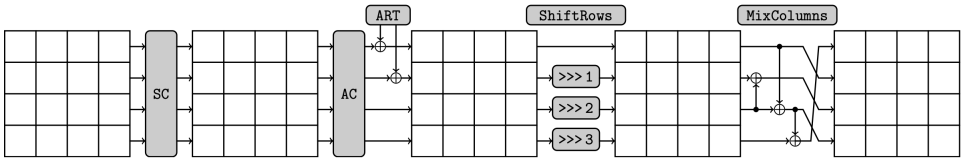


Figure 1: The SKINNY round function applies five different transformations: SubCells (SC), AddConstants (AC), AddRoundTweakey (ART), ShiftRows (SR) and MixColumns (MC) as depicted in Beierle *et al.* [BJK+16]

4.2 SKINNY Quadratic Function With Fewer Random Bits

While a straightforward secure computation of the quadratic class $\mathcal{Q}_{294}$ would typically require two sequential masked Toffoli gates, we propose a specific optimization for our two-share, third-order secure assembly implementation evaluated in Section 5. Our approach only requires nine bits of randomness, outperforming an equivalent implementation of the quadratic class using Boolean masking in terms of both the number of AND gates and required bits of randomness. Moreover, the randomness can be used as-is and does not need to stem from a secure code-based encoding of zero $CB(0)$. Hence, its generation does not induce extra cost.

In our implementation, any 4-bit unshared value $x = (a, b, c, d) \in \mathbb{F}_2^4$ (where a is the least significant bit) is masked using the public matrix $\mathbf{A} = [\mathbf{G}; \mathbf{H}]$ as detailed in Section 3.1.1 in order to obtain the sharing $\bar{x} = (x_0, x_1)$ with $x_i = (a_i, b_i, c_i, d_i) \in \mathbb{F}_2^4$.

The selection of the matrix $\mathbf{A}$ for encoding sensitive variables is guided by its maximal dual distance, designed specifically for our two-share, four-bit input scenario, ensuring third-order bit-level security of the encodings, as detailed in [CGC+21, CLGR24]. The optimization relies on two observations. First, note that the multiplication computed in both Toffoli gates $a + bd$ and $b + cd$ of $\mathcal{Q}_{294}$ are done with the same value d. Second, with the generating matrix $\mathbf{A}$ as described above, half of the binary shares of b and c are identical. Let us rewrite the multiplications using their corresponding binary shares as $bd = (b_0 + a_1 + c_1 + d_1)(d_0 + a_1 + b_1 + c_1)$ and as $cd = (c_0 + a_1 + b_1 + d_1)(d_0 + a_1 + b_1 + c_1)$. Hence, we note the encoding of sensitive bits b and c both have in common the two shares a_1 and d_1. Thus, the eight cross products from $(a_1 + d_1)(d_0 + a_1 + b_1 + c_1)$ can be computed once and re-used for both Toffoli gates. This optimization is made possible by the code-based masking which shares some randomness in the encoding of different sensitive bits.

Although the algorithms introduced in Section 3 offer the advantage of generality, they depend on specific randomness (*e.g.*, shares of code-based of zero) during the re-masking of the cross products, which incurs additional costs. The optimized version requires only nine bits of randomness denoted by $\{z_0, z_1, z_2\}, \{r_0, ..., r_5\}$. However, this efficiency comes at the cost of requiring a specific order for executing the cross product computations and placing the randomness.

The optimized computation of $\mathcal{Q}_{294}$ is detailed below. It follows a similar methodology as in Section 3.1 where the original encoding $\bar{x}$ is first refreshed and then, selected shares are successively re-masked and added together to compute the output. Note that in order

to optimize randomness cost, only the binary shares of the bit d are actually refreshed and not the whole encoding $\bar{x}$.

We denote the input sharing by $\bar{x} = (a_0, b_0, c_0, d_0, a_1, b_1, c_1, d_1)$ and the output sharing by $\bar{z} = (s_0, w_0, y_0, t_0, s_1, w_1, y_1, t_1)$. Note that the order of the operations is important and should be executed from left to right. First we refresh the shares of the sensitive value d. We make the refreshed of d'_0, a'_1, b'_1, c'_1 as follows

$$\begin{aligned} d'_0 &= d_0 + r_1 + r_4 + z_1 + z_0 + z_2 + r_1 + r_4 \\ a'_1 &= a_1 + z_0 \\ b'_1 &= b_1 + z_1 \\ c'_1 &= c_1 + z_2 \end{aligned}$$

Next, we calculate the cross products in the following order with the left branch giving the output s_0 and the right branch giving w_0

$$\begin{aligned} & a_1 d'_0 + r_0 \\ & d_1 d'_0 + r_1 \\ & a_1 a'_1 \\ & d_1 a'_1 + r_2 \\ & a_1 b'_1 \\ & d_1 b'_1 \\ & a_1 c'_1 \\ & d_1 c'_1 + r_3 \end{aligned}$$

$\downarrow$ $\qquad\qquad$ $\downarrow$

$b_0 d'_0$	$c_0 d'_0$
$c_1 d'_0 + r_4$	$b_1 d'_0 + r_5$
$b_0 a'_1$	$c_0 a'_1$
$c_1 a'_1 + r_0$	$b_1 a'_1 + r_2$
$b_0 b'_1$	$c_0 b'_1$
$c_1 b'_1$	$b_1 b'_1$
$b_0 c'_1 + r_2$	$c_0 c'_1 + r_1$
$c_1 c'_1$	$b_1 c'_1$
a_0	b_0
r_0	

We end up with the output sharing as follows

$$\begin{aligned} s_0 &= a_0 + bd + r_1 + r_3 + r_4 + r_0 & \qquad s_1 &= a_1 + r_1 + r_3 \\ w_0 &= b_0 + cd + r_0 + r_3 + r_5 & \qquad w_1 &= b_1 + r_5 + r_4 + r_3 \\ y_0 &= c_0 + r_0 + r_5 & \qquad y_1 &= c_1 + r_4 + r_5 \\ t_0 &= d_0 + r_1 & \qquad t_1 &= d_1 + r_0 + r_4 + r_1 \end{aligned}$$

which decodes to $(a + bd, b + cd, c, d)$.

4.3 Security

The optimized gadget above is not trivially composable secure. However, it suffices to protect the SKINNY cipher.

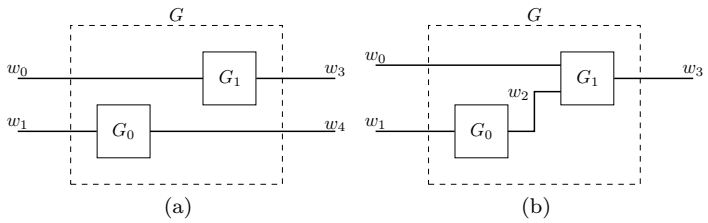


Figure 2: Sequential compositions of G_0 and G_1 denoted by G.

We show that the combination of a probing secure, uniform, and PINI secure gadget allows for compositions such that the composed design is again secure. In other words, while PINI is a composable security notion in the sense that the arbitrary composition between PINI secure gadgets again provides a gadget that achieves the same notion of security, so does the sequential composition (as shown in Figure 2) of gadgets that are uniform, probing secure, and PINI create a new gadget that achieves the same security notion. Thus, the combination of these security notions is a "sequential composable security notion". To be clear, it is not probing security which achieves this composable notion of security but the combination of probing security with uniformity and the PINI notion. To prove this, we need the following theorem.

Theorem 3. *The sequential composition of uniform, t-PINI, and $t+1$ probing secure gadgets is again uniform, t-PINI, and $t+1$ probing secure.*

Proof. Take two gadgets G_0 and G_1 which are sequentially composed and are both t-PINI, $t+1$ probing secure, and uniform. Call their composition G. We consider the two most general possibilities for the composition as shown in Figure 2 (where in the second figure w_0 can also be removed). From [DNR19], we know that uniformity is sequentially composable and from [CS20], we also know that 2-PINI is trivially composable, so G is again uniform and 2-PINI. We then show that G is $t+1$ probing secure when G is given a uniform input. Since the probing security is evident for the parallel composition in Figure 2, we focus on the second composition given in the figure. We consider that there are t'_0 probes placed in G_0 and t'_1 probes in G_1 such that $t'_0 + t'_1 = t' \leq t+1$. In case all probes are placed in one gadget, $t'_0 = t+1$ or $t'_1 = t+1$, then the security follows from each gadget being given a uniform input sharing and G_0 or G_1 being $t+1$ probing secure. Otherwise, $t'_0, t'_1 \leq t$. In this case, since G_1 is t-PINI, the t'_1 probes can be simulated using t'_1 input shares of w_2. Then, since G_0 has a uniform input and is $t+1$ probing secure, the t'_0 intermediate probes and the t'_1 shares of w_2 can simulated from scratch showing that G is also $t+1$ probing secure.

□

Note that Theorem 3 does not imply that stand-alone probing security is inherently composable, nor does it guarantee direct applicability to the composition of probing-secure sub-circuits. However, the above theorem is quite interesting as it is a general result showing that in a *limited setting of composability* (eg., sequential composability), the notion of uniformity allows to use a less strict notion of non-interference (namely one order down). For $t = 1$, the above theorem provides the security of "threshold implementations" by Nikova *et al.* [NRR06] where uniformity provides sequential composable security and no non-interference notion is needed.

We now focus on the security of the sharing of the $\mathcal{Q}_{294}$ function. Via a brute-force verification using software, we verify that this function is uniform, third-order probing secure, and 2-PINI. This provides the following argument of security for the SKINNY masking.

Theorem 4. *The two-share SKINNY with the $\mathcal{Q}_{294}$ sharing in Section 4.2 is third-order probing secure.*

Proof. First, recall that the $\mathcal{Q}_{294}$ sharing is uniform, 2-PINI, and third-order probing secure. Since the masking of SKINNY consists of the 16-parallel composition of S-boxes which each consists of the sequential composition of the $\mathcal{Q}_{294}$ sharing and invertible bitwise affine layers, the entire S-box masking layer is uniform, 2-PINI, and third-order probing secure following Theorem 3. Finally, since the linear layer from SKINNY is also 2-PINI and third-order probing secure (since it works share-wise using only one bit per sharing), its composition with the nonlinear layer remains third-order probing secure and 2-PINI. Thus, the composition of all masked rounds of SKINNY is third-order secure following Theorem 3. □

Consequently, the two-share code-based masked SKINNY implementation, employing the sharing scheme detailed in Section 4.2, achieves bit-probing security (or, equivalently, bounded moment security) under linear leakages at order 3, in contrast to a first-order word-level security.

4.4 Algorithmic Efficiency

We complement our analysis by evaluating the algorithmic efficiency in terms of the number of XORs, ANDs, and random bits required to mask the third-order secure $\mathcal{Q}_{294}$ function using code-based masking with general Algorithm-1 (TOF), the optimized computation from Section 4.2, and the ISW Boolean masking. The costs are detailed in Table 2. Boolean masking follows a standard four-share masked Toffoli gate computation where each masked cross product (*e.g.*, $bd = (b_0 \oplus b_1 \oplus b_2 \oplus b_3)(d_0 \oplus d_1 \oplus d_2 \oplus d_3)$) requires 6 random bits. As previously mentioned, the main benefit of the algorithms in Section 3 is their generality and versatility for securely computing any masked function over code-based sharing. However, this advantage comes at the cost of an increased number of XOR gates and fresh randomness. Finally, the tailored implementation of the optimized computation demonstrates the best results, with a smaller gate count and reduced randomness requirements compared to the other implementations. Moreover, we gain an additional speed-up for the key-addition and the linear layer of SKINNY since these are still achieved by using simple operations for code-based masking, however, the state size is twice smaller compared to the equivalently secure Boolean masked variant.

Table 2: Comparison of the cost of masking $\mathcal{Q}_{294}$. Note that the cost of Algorithm 1 does not take into account the gates count for the $CB(0)$ computations.

	#ANDs	#XORs	#Rand
Boolean Masking	32	56	12
Optimized $\mathcal{Q}_{294}$	24	58	9
Algorithm 1	32	288	128

5 Practical Side-Channel Leakage Evaluation

This section presents the practical side-channel leakage evaluation of our optimized bitsliced SKINNY implementation detailed in Section 4. We confirm the theoretically predicted third-order security of the code-based bitsliced implementation from Theorem 4 on a real-world device. We first detail the cipher and its assembly implementation details. Next we show our evaluation on a Cortex-M4 microcontroller.

5.1 Implementation

To effectively manage the use of the registers and optimize the implementation's running time, the masked SKINNY gadget is fully implemented in ARM assembly. For simplicity, our implementation targets SKINNY-64-64 (*e.g.*, using a 64-bit block and tweakey) but could be extended to other configurations of the cipher at the cost of increased overhead (due to additional computation of the tweakey schedule). Our implementation follows a bitslice structure where each register r_j is filled with the j^{th} bit of each 16 cells in the state. The bitslice structure is made from the received shares at the start of the cipher and reconstructed back into a word at the end of the cipher. The masking is computed with the generating matrices $\mathbf{A} = [\mathbf{G}; \mathbf{H}]$ as described in Section 4.2.
In order to practically maintain security order amplification enabled with code-based masking, some adjustments in the assembly implementation have to be made to avoid certain micro-architectural leakages. Previous works describe more complex masking types as less exposed to transition-based leakages compared to Boolean masking [BFG+17]. In a Boolean encoding where $x = x_0 + x_1$, transition-based leakages expose not only noisy versions of x_0 and x_1 but also their distance to the previous elements on the same hardware resources. For instance, if x_1 overwrites the x_0 share, with Hamming distance leakages, the adversary learns some noisy value of $HW(x)$, effectively nullifying the benefits of secret sharing. This issue commonly arises in microcontrollers when a register stores consecutive shares of the same variable, reducing security. However, code-based masking improves resistance to such leakages over word elements. Indeed, given the encoding of x as $[x, r_x][\mathbf{G}; \mathbf{H}]$, the Hamming distance observed by an adversary due to transition leakage—such as when x_0 is overwritten by x_1 in a register—remains uniformly distributed for $\mathbf{H} \neq \mathbf{I}$, where $\mathbf{I}$ is the identity matrix, similarly to what is observed for the specific case of inner product masking [BFG+17].

However, we observe that the bit-level security order remains vulnerable to reduction in the presence of transition-based leakages. Such a reduction is expected to follow the order-reduction theorem from Balasch *et al.* [BGG+14, Theorem 1] stating that a d^{th}-order secure masking scheme under value-based leakages is $\lfloor \frac{d}{2} \rfloor^{\text{th}}$-order secure under distance-based leakages. Indeed, we note that on a bit-level, each sensitive bit of a word can be seen as masked as a d-order Boolean masking. For instance, the first sensitive bit in our implementation is encoded "as a" third-order masking, where $a = a_0 \oplus b_1 \oplus c_1 \oplus d_1$. Thus, the order-reduction theorem similarly applies to each individual bit. In practical scenarios, transition-based leakages often arise from register-based transitions. Therefore, a bitslice implementation may amplify the impact of such leakages. For example, assuming shares as described in Section 4.2, transition leakages between shares $(a_0, b_0) = a + b + a_1 + b_1$ and shares $(a_1, b_1) = a_1 + b_1$ result into a reduction from third to first order security. Such effects especially impact software implementations where each assembly instruction is computed consecutively. Hence, countermeasures have to be applied to preserve practical bit-level security order. We choose to use the share shifting methodology and to follow the security properties proposed in Gaspoz *et al.* [GD23] namely: vertical non-completeness, horizontal non-completeness and register uniformity. In the original methodology, in order to ensure vertical non-completeness, shares from similar domains are kept aligned and the ones from different domains are shifted by a value. However, since our security level is described over bits, each share $(a_0, \ldots, d_0, a_1, \ldots, d_1)$ must now be shifted by a unique value (*e.g.*, no shares can be aligned). Figure 3 illustrates how the shares are placed within the register file in our implementation. Namely, from the sharing $\bar{x} \in \mathbb{F}_2^8$ visualised as $\bar{x} = (x_0, x_1)$ where $x_i = (a_i, b_i, c_i, d_i) \in \mathbb{F}_2^4$, each row defines a half-full 32-bit word register filled with bits labelled as a_i^j (resp. d_i^j) to represent the least significant bit (resp. most significant bit) of the i^{th} share of the j^{th} S-Box. Thus, both linear and non-linear operations require realignment of the shares at a unique index (*e.g.*, starting from index 8 and above). Note that only half of the register space is utilized for computing one state

block. Thus, additional optimization could allow for simultaneous computation of at most two blocks.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\mathbf{a_0^0}$	a_0^1	a_0^2	a_0^3	a_0^4	a_0^5	a_0^6	a_0^7	a_0^8	a_0^9	a_0^{10}	a_0^{11}	a_0^{12}	a_0^{13}	a_0^{14}	a_0^{15}
b_0^{15}	$\mathbf{b_0^0}$	b_0^1	b_0^2	b_0^3	b_0^4	b_0^5	b_0^6	b_0^7	b_0^8	b_0^9	b_0^{10}	b_0^{11}	b_0^{12}	b_0^{13}	b_0^{14}
c_0^{14}	c_0^{15}	$\mathbf{c_0^0}$	c_0^1	c_0^2	c_0^3	c_0^4	c_0^5	c_0^6	c_0^7	c_0^8	c_0^9	c_0^{10}	c_0^{11}	c_0^{12}	c_0^{13}
d_0^{13}	d_0^{14}	d_0^{15}	$\mathbf{d_0^0}$	d_0^1	d_0^2	d_0^3	d_0^4	d_0^5	d_0^6	d_0^7	d_0^8	d_0^9	d_0^{10}	d_0^{11}	d_0^{12}
a_1^{12}	a_1^{13}	a_1^{14}	a_1^{15}	$\mathbf{a_1^0}$	a_1^1	a_1^2	a_1^3	a_1^4	a_1^5	a_1^6	a_1^7	a_1^8	a_1^9	a_1^{10}	a_1^{11}
b_1^{11}	b_1^{12}	b_1^{13}	b_1^{14}	b_1^{15}	$\mathbf{b_1^0}$	b_1^1	b_1^2	b_1^3	b_1^4	b_1^5	b_1^6	b_1^7	b_1^8	b_1^9	b_1^{10}
c_1^{10}	c_1^{11}	c_1^{12}	c_1^{13}	c_1^{14}	c_1^{15}	$\mathbf{c_1^0}$	c_1^1	c_1^2	c_1^3	c_1^4	c_1^5	c_1^6	c_1^7	c_1^8	c_1^9
d_1^9	d_1^{10}	d_1^{11}	d_1^{12}	d_1^{13}	d_1^{14}	d_1^{15}	$\mathbf{d_1^0}$	d_1^1	d_1^2	d_1^3	d_1^4	d_1^5	d_1^6	d_1^7	d_1^8

Figure 3: Placement of shares in the register file of the bitsliced implementation.

5.2 Measurement Setup

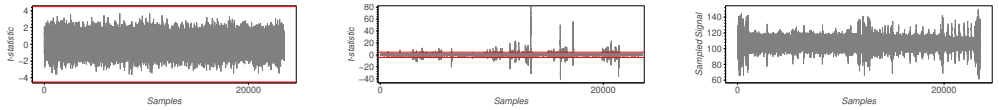
Measurement tests were conducted using a CW308T-STM32F target board, which features an STM32F415RG Cortex-M4 microcontroller operating at an internal frequency of 24 MHz derived from an external 8 MHz clock. Traces were obtained from the NewAE CW308 UFO board using a Tektronix DPO70404C oscilloscope with a sample rate of 125 MS/s. All measurements were synchronized by aligning the oscilloscope with the external clock. Finally, the Arm GNU toolchain[2] was used for compiling the Cortex-M4 assembly implementation.

5.3 Leakage Assessment

The side channel evaluation leverages the well-known Test Vector Leakage Assessment (TVLA) methodology first introduced in [CDMG+13]. We perform the evaluation in a non-specific, fixed vs. random configuration where traces are partitioned in two sets, a first set S_0 which results from fixed plaintexts and a second set S_1 which corresponds to random plaintexts given in a random fashion to the masked SKINNY implementation. Next, the t-test distinguisher is computed on every sample point to assess the validity of the null hypothesis which claims that the two sets share the same first-order moment. In the literature, the threshold value to determine if the null hypothesis is rejected is typically set at $t = \pm 4.5$, corresponding to a confidence level of approximately 99.999%. Thus, values within the threshold t indicate that the sets of measurements are indistinguishable from each other, demonstrating that the masking countermeasure is effective. Figure 4c illustrates the mean trace of a full round of the masked implementation containing 23500 sample points. Figure 4a shows the first-order evaluation of the first round of our masked SKINNY implementation for one million traces. For completeness, Figure 4b shows first-order evaluation with mask off which leaks as expected. Since we anticipate third-order security, first-order security is obviously expected. We recall that first-order two-share software-masked code-based encoding implementations enjoy inherent protection against transition leakage. This is due to the uniformly distributed Hamming distance between the two shares [BFG+17, Section 6.3], a benefit that is often challenging to achieve with Boolean masking using just two shares. We then expand our leakage assessment to multivariate higher order evaluations using the SCALib [CB23] library. In this evaluation, each unique combination of two sample points in the trace is first combined using a pre-processing function (*i.e.*, the centered product [PRB09]) followed by a t-test evaluation

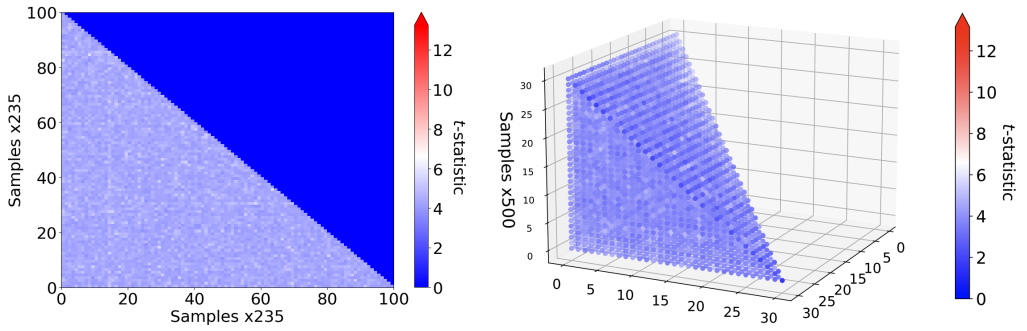
[2] gcc-arm-11.2-2022.02-x86_64-arm-none-eabi

at each computed point. As the number of sample points to evaluate increases, it becomes necessary to adjust the t threshold accordingly to avoid false positives [DZD+17] that would result by keeping the usual $t \pm 4.5$ value. Figure 5a illustrates the bivariate evaluation of one round of the SKINNY ARM assembly implementation using one million traces, with no second-order leakages detected. While second-order bivariate evaluations were performed using the full length of the captured traces, third-order trivariate evaluations pose a significant computational challenge due to the impractical number of sample points that need to be evaluated. Therefore, we limit our evaluation to the SKINNY S-Box computation for which we further downsample the captured traces keeping only a subset of one twentieth sample points, before doing the preprocessing. Thus leading to a trace of 755 sample points to be evaluated at third order. Figure 5b shows the results of our trivariate evaluation, which aligns with our theoretical expectations from Theorem 4 as no significant evidence of leakage was detected for one million measurements.



(a) First-order mask on (b) First-order mask off (c) Mean

Figure 4: First-order t-test *non-specific*, *fixed vs. random* results of one round of the SKINNY ARM assembly. The ± 4.5 threshold is marked by red lines. Experiments with mask on and off use 1M and 10k traces, respectively.



(a) Second-order bivariate t-test (b) Third-order trivariate t-test

Figure 5: Second-order and third-order multivariate t-test *non-specific*, *fixed vs. random* results of one round of the SKINNY ARM assembly. Second-order bivariate test is done on the first round of SKINNY. Third-order trivariate test is done over the S-Box. Experiments are done using 1M traces.

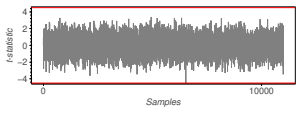
5.4 Cost and Cycle Count

Table 3 presents the costs associated with executing the third-order secure assembly implementation of SKINNY-64-64. Our implementation completes in approximately 5 milliseconds. As shown in Table 2, each S-Box requires 18 bits of randomness. With 32 rounds of execution, the code-based implementation accumulates a total of 9216 random bits. As expected, the non-linear layer accounts for the majority of the cycle count per round. To the best of our knowledge, no other masked implementations of the SKINNY cipher using different masking schemes currently exist let alone at third-order security.

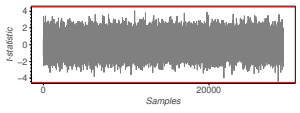
However, to ensure a fair comparison, we provide two bitsliced Boolean masking assembly implementations of SKINNY: one with two shares ($d = 1$) and another with four shares ($d = 3$). Similarly to the code-based implementation, and in order to reduce the impact transition leakage, the Boolean masking implementations follow a similar methodology of shares shifting as detailed in [GD23]. The cross products of the quadratic class $\mathcal{Q}_{294}$ are computed using ISW multiplication which requires $d(d+1)/2$ random bits per cross products. Additionally, Figure 6 presents the practical security evaluations of the two Boolean masking implementations. We observe that while the two-share implementation remains first-order secure, the four-share implementation exhibits a single leakage point in the bivariate analysis and no leakage in its trivariate evaluation (due to noise). It is important to note that achieving fully secure Boolean masking is beyond the scope of this work; however, our results establish the minimum security metrics required for such an implementation (as more instructions would be required to patch it). The two-share Boolean masking implementation obviously computes in a shorter time and uses fewer random bits than the two-share code-based implementation. However, unlike our code-based masking implementation, the Boolean masking implementation does not benefit from security order amplification and remains restricted to first-order security. Finally, a more relevant comparison is between our two-share code-based implementation and the four-share BM implementation, as both target third-order security. In this comparison, we find that the Boolean masking implementation incurs up to 1.4× higher cycle counts and consumes 1.3× more random bits than our code-based implementation.

Table 3: Performance results for 32 rounds of SKINNY-64-64, comparing our code-based (CB) masked assembly implementation with Boolean masking (BM) implementations in assembly. Time is given in milliseconds, ROM and RAM are given in bytes.

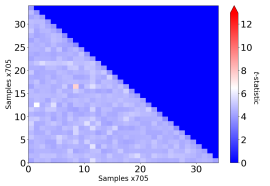
Variant	Shares	Time	Cycles	ROM	RAM	Random Bits
CB SKINNY-64-64	2	5.063	121508	43376	2840	9216
BM SKINNY-64-64	2	2.728	65443	13880	1260	2048
BM SKINNY-64-64	4	7.221	173164	25960	4400	12288



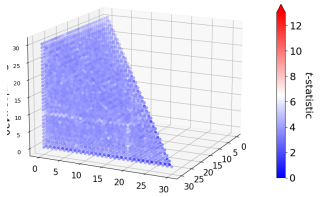
(a) First-order BM 2-share



(b) First-order BM 4-share



(c) Second-order bivariate BM 4-share



(d) Third-order trivariate BM 4-share

Figure 6: First-order, second-order, and third-order multivariate t-test *non-specific*, *fixed vs. random* results of one round of the SKINNY Boolean masking assembly implementations. First- and second-order bivariate tests are done on the first round of SKINNY. The third-order trivariate test is done over the S-Box. Experiments are conducted using 1M traces.

6 Conclusion and Future Work

In this work, we introduced a generalized methodology for computing any masked function using code-based encodings while preserving the bit-probing security order during com-

putation. Specifically, we detailed the construction of basic composable masked circuits using linear and Toffoli gates, which can be used to build larger masked circuits. This enables, for the first time, the application of code-based encodings to any cipher operating on bits, benefiting from the security order amplification effect (e.g., bit-probing security order) of code-based masking. Furthermore, this approach allows for bitslice code-based implementations, reducing the computational cost associated with more complex masking schemes and previously limited to Boolean masking. We then presented an optimized third-order secure tailored Cortex-M4 ARM assembly implementation of the SKINNY cipher that outperforms its Boolean masking counterpart in terms of gate and randomness requirements. Finally, we validated our theoretical security order expectations with a practical side-channel leakage evaluation of our ARM SKINNY assembly implementation, where no significant evidence of leakage was detected across one million measurements. The application of code-based masking to the SKINNY cipher was motivated by its simple permutation layer. Unlike Boolean masking, where masked permutation layers are straightforward and cost-effective to construct, code-based masked permutation layers can become cumbersome. For instance, in the PRESENT cipher, an unexpected overhead would occur at the masked permutation layer due to the shuffling of bits over the 64-bit state. Therefore, reducing the overhead at such linear layers is an interesting area for future research. Additionally, the generalized algorithms depend on the computation of secure code-based encoding of zero (*e.g.*, $CB(0)$), which is not computationally free, unlike randomness in Boolean masking. As a result, we propose as future work the development of more optimized generalized bit-t-PINI secure basic circuits or less expensive secure code-based encodings of zero ($CB(0)$).

Acknowledgment. We thank Vincent Rijmen, Svetla Nikova and Ventzislav Nikov for the interesting discussions. This work was supported by CyberSecurity Research Flanders with reference number VR20192203. Siemen Dhooghe is supported by a PhD Fellowship from the Research Foundation – Flanders (FWO).

References

[APB+23] Samuele Andreoli, Enrico Piccione, Lilya Budaghyan, Pantelimon Stanica, and Svetla Nikova. On decompositions of permutations in quadratic functions. *IACR Cryptol. ePrint Arch.*, page 1632, 2023.

[BCC+14] Julien Bringer, Claude Carlet, Hervé Chabanne, Sylvain Guilley, and Houssem Maghrebi. Orthogonal direct sum masking - A smartcard friendly computation paradigm in a code, with builtin protection against side-channel and fault attacks. In David Naccache and Damien Sauveron, editors, *Information Security Theory and Practice. Securing the Internet of Things - 8th IFIP WG 11.2 International Workshop, WISTP 2014, Heraklion, Crete, Greece, June 30 - July 2, 2014. Proceedings*, volume 8501 of *Lecture Notes in Computer Science*, pages 40–56. Springer, 2014.

[BCP97] Wieb Bosma, John Cannon, and Catherine Playoust. The Magma algebra system. I. The user language. *J. Symbolic Comput.*, 24(3-4):235–265, 1997. Computational algebra and number theory (London, 1993).

[BCRT23] Sonia Belaïd, Gaëtan Cassiers, Matthieu Rivain, and Abdul Rahman Taleb. Unifying freedom and separation for tight probing-secure composition. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*,

volume 14083 of *Lecture Notes in Computer Science*, pages 440–472. Springer, 2023.

[BDF+16] Gilles Barthe, François Dupressoir, Sebastian Faust, Benjamin Grégoire, François-Xavier Standaert, and Pierre-Yves Strub. Parallel implementations of masking schemes and the bounded moment leakage model. *IACR Cryptol. ePrint Arch.*, page 912, 2016.

[BFG+17] Josep Balasch, Sebastian Faust, Benedikt Gierlichs, Clara Paglialonga, and François-Xavier Standaert. Consolidating inner product masking. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 724–754. Springer, 2017.

[BGG+14] Josep Balasch, Benedikt Gierlichs, Vincent Grosso, Oscar Reparaz, and François-Xavier Standaert. On the cost of lazy engineering for masked software implementations. In Marc Joye and Amir Moradi, editors, *Smart Card Research and Advanced Applications - 13th International Conference, CARDIS 2014, Paris, France, November 5-7, 2014. Revised Selected Papers*, volume 8968 of *Lecture Notes in Computer Science*, pages 64–81. Springer, 2014.

[BJK+16] Christof Beierle, Jérémy Jean, Stefan Kölbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The SKINNY family of block ciphers and its low-latency variant MANTIS. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology - CRYPTO 2016 - 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II*, volume 9815 of *Lecture Notes in Computer Science*, pages 123–153. Springer, 2016.

[CB23] Gaëtan Cassiers and Olivier Bronchain. Scalib: A side-channel analysis library. *Journal of Open Source Software*, 8(86):5196, 2023.

[CDMG+13] J. Cooper, E. De Mulder, G. Goodwill, J. Jaffe, G. Kenworthy, and P. Rohatgi. Test vector leakage assessment (TVLA) methodology in practice. http://icmc-2013.org/wp/wp-content/uploads/2013/09/goodwillkenworthtestvector.pdf, 2013.

[CGC+21] Wei Cheng, Sylvain Guilley, Claude Carlet, Sihem Mesnager, and Jean-Luc Danger. Optimizing inner product masking scheme by a coding theory approach. *IEEE Trans. Inf. Forensics Secur.*, 16:220–235, 2021.

[CGM20] Claude Carlet, Sylvain Guilley, and Sihem Mesnager. Direct sum masking as a countermeasure to side-channel and fault injection attacks. *IACR Cryptol. ePrint Arch.*, page 876, 2020.

[CJRR99] Suresh Chari, Charanjit S. Jutla, Josyula R. Rao, and Pankaj Rohatgi. Towards sound approaches to counteract power-analysis attacks. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 398–412. Springer, 1999.

[CLGR24] Wei Cheng, Yi Liu, Sylvain Guilley, and Olivier Rioul. Toward finding best linear codes for side-channel protections (extended version). *J. Cryptogr. Eng.*, 14(1):131–145, 2024.

[CS20] Gaëtan Cassiers and François-Xavier Standaert. Trivially and efficiently composing masked gadgets with probe isolating non-interference. *IEEE Trans. Inf. Forensics Secur.*, 15:2542–2555, 2020.

[DDE+20] Joan Daemen, Christoph Dobraunig, Maria Eichlseder, Hannes Groß, Florian Mendel, and Robert Primas. Protecting against statistical ineffective fault attacks. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(3):508–543, 2020.

[DNR19] Siemen Dhooghe, Svetla Nikova, and Vincent Rijmen. Threshold implementations in the robust probing model. In *TIS@CCS*, pages 30–37. ACM, 2019.

[DZD+17] A. Adam Ding, Liwei Zhang, François Durvaux, François-Xavier Standaert, and Yunsi Fei. Towards sound and optimal leakage detection procedure. In Thomas Eisenbarth and Yannick Teglia, editors, *Smart Card Research and Advanced Applications - 16th International Conference, CARDIS 2017, Lugano, Switzerland, November 13-15, 2017, Revised Selected Papers*, volume 10728 of *Lecture Notes in Computer Science*, pages 105–122. Springer, 2017.

[GD23] John Gaspoz and Siemen Dhooghe. Threshold implementations in software: Micro-architectural leakages in algorithms. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2023(2):155–179, 2023.

[GD24] John Gaspoz and Siemen Dhooghe. Bit t-SNI secure multiplication gadget for inner product masking. Cryptology ePrint Archive, Paper 2024/1546, 2024.

[GM11] Louis Goubin and Ange Martinelli. Protecting AES with shamir's secret sharing scheme. In Bart Preneel and Tsuyoshi Takagi, editors, *Cryptographic Hardware and Embedded Systems - CHES 2011 - 13th International Workshop, Nara, Japan, September 28 - October 1, 2011. Proceedings*, volume 6917 of *Lecture Notes in Computer Science*, pages 79–94. Springer, 2011.

[GP99] Louis Goubin and Jacques Patarin. DES and differential power analysis (the "duplication" method). In Çetin Kaya Koç and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems, First International Workshop, CHES'99, Worcester, MA, USA, August 12-13, 1999, Proceedings*, volume 1717 of *Lecture Notes in Computer Science*, pages 158–172. Springer, 1999.

[IKL+13] Yuval Ishai, Eyal Kushilevitz, Xin Li, Rafail Ostrovsky, Manoj Prabhakaran, Amit Sahai, and David Zuckerman. Robust pseudorandom generators. In *ICALP (1)*, volume 7965 of *Lecture Notes in Computer Science*, pages 576–588. Springer, 2013.

[ISW03] Yuval Ishai, Amit Sahai, and David A. Wagner. Private circuits: Securing hardware against probing attacks. In Dan Boneh, editor, *Advances in Cryptology - CRYPTO 2003, 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 2003, Proceedings*, volume 2729 of *Lecture Notes in Computer Science*, pages 463–481. Springer, 2003.

[NRR06] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold implementations against side-channel attacks and glitches. In Peng Ning, Sihan Qing, and Ninghui Li, editors, *Information and Communications Security,*

8th International Conference, ICICS 2006, Raleigh, NC, USA, December 4-7, 2006, Proceedings, volume 4307 of *Lecture Notes in Computer Science*, pages 529–545. Springer, 2006.

[PGS+17] Romain Poussier, Qian Guo, François-Xavier Standaert, Claude Carlet, and Sylvain Guilley. Connecting and improving direct sum masking and inner product masking. In Thomas Eisenbarth and Yannick Teglia, editors, *Smart Card Research and Advanced Applications - 16th International Conference, CARDIS 2017, Lugano, Switzerland, November 13-15, 2017, Revised Selected Papers*, volume 10728 of *Lecture Notes in Computer Science*, pages 123–141. Springer, 2017.

[PGS+18] Romain Poussier, Qian Guo, François-Xavier Standaert, Claude Carlet, and Sylvain Guilley. Connecting and improving direct sum masking and inner product masking. In Thomas Eisenbarth and Yannick Teglia, editors, *Smart Card Research and Advanced Applications*, pages 123–141, Cham, 2018. Springer International Publishing.

[PRB09] Emmanuel Prouff, Matthieu Rivain, and Régis Bevan. Statistical analysis of second order differential power analysis. *IEEE Trans. Computers*, 58(6):799–811, 2009.

[SM21] Aein Rezaei Shahmirzadi and Amir Moradi. Second-order SCA security with almost no fresh randomness. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(3):708–755, 2021.

[WCG+22] Qianmei Wu, Wei Cheng, Sylvain Guilley, Fan Zhang, and Wei Fu. On efficient and secure code-based masking: A pragmatic evaluation. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(3):192–222, 2022.

[WMCS20] Weijia Wang, Pierrick Méaux, Gaëtan Cassiers, and François-Xavier Standaert. Efficient and private computations with code-based masking. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(2):128–171, 2020.

[WSY+16] Weijia Wang, François-Xavier Standaert, Yu Yu, Sihang Pu, Junrong Liu, Zheng Guo, and Dawu Gu. Inner product masking for bitslice ciphers and security order amplification for linear leakages. In Kerstin Lemke-Rust and Michael Tunstall, editors, *Smart Card Research and Advanced Applications - 15th International Conference, CARDIS 2016, Cannes, France, November 7-9, 2016, Revised Selected Papers*, volume 10146 of *Lecture Notes in Computer Science*, pages 174–191. Springer, 2016.

[WYS19] Weijia Wang, Yu Yu, and François-Xavier Standaert. Provable order amplification for code-based masking: How to avoid non-linear leakages due to masked operations. *IEEE Trans. Inf. Forensics Secur.*, 14(11):3069–3082, 2019.